

CS3002E Compiler Design

Monsoon 2025

Lecture #12

Bottom-Up Parsing - 1

Department of Computer Science & Engineering
NIT Calicut

Top-down Parsing: Review

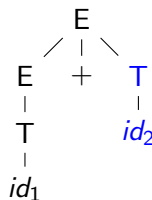
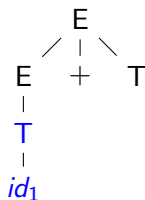
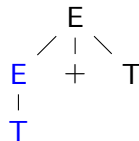
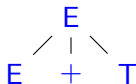
$$E \rightarrow E + T \mid T$$

$$T \rightarrow \mathbf{id}$$

Build a parse tree top-down for $id_1 + id_2$

Building parse tree top-down for $id_1 + id_2$

E



Top-down Parsing: Review

$$E \rightarrow E + T \mid T$$

$$T \rightarrow \mathbf{id}$$

Do a leftmost derivation of $id_1 + id_2$

Top-down Parsing: Review

$$E \rightarrow E + T \mid T$$

$$T \rightarrow \mathbf{id}$$

Leftmost derivation of $id_1 + id_2$

$$E \Rightarrow E + T \Rightarrow T + T \Rightarrow id_1 + T \Rightarrow id_1 + id_2$$

Top-down Parsing: Review

$$E \rightarrow E + T \mid T$$

$$T \rightarrow \mathbf{id}$$

Leftmost derivation of $id_1 + id_2$

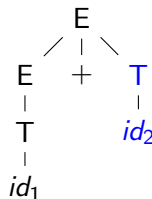
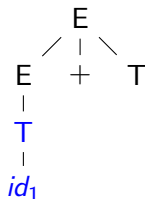
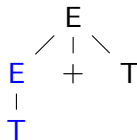
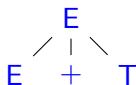
$$E \Rightarrow E + T \Rightarrow T + T \Rightarrow id_1 + T \Rightarrow id_1 + id_2$$

Relate the derivation steps with the steps in building the parse tree.

Top-down parsing: Leftmost derivation

$$E \Rightarrow E + T \Rightarrow T + T \Rightarrow id_1 + T \Rightarrow id_1 + id_2$$

E



The *yield* of each tree is a leftmost sentential form.

Bottom-up Parsing

- Simulates the steps in building a parse tree bottom-up
 - starting from the leaf nodes, working up towards the root
- Equivalently does a rightmost derivation in reverse

Building Parse Tree Bottom-up

$E \rightarrow E + T \mid T$

$T \rightarrow \mathbf{id}$

$id_1 + id_2$

$T + id_2$
|
 id_1

$E + id_2$
|
 T
|
 id_1

Building Parse Tree Bottom-up

$id_1 + id_2$

$T + id_2$
|
 id_1

$E + id_2$
|
 T
|
 id_1

$E + T$
| |
 T id_2
|
 id_1

E
/ | \
 E $+$ T
| |
 T id_2
|
 id_1

Rightmost Derivation

$$E \rightarrow E + T \mid T$$

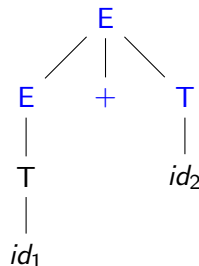
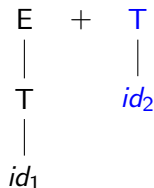
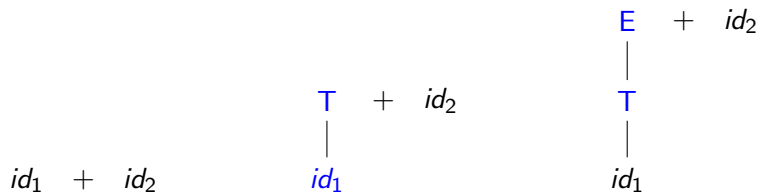
$$T \rightarrow \mathbf{id}$$

Rightmost derivation of $id_1 + id_2$

$$E \Rightarrow E + T \Rightarrow E + id_2 \Rightarrow T + id_2 \Rightarrow id_1 + id_2$$

Bottom-up parsing: relating to rightmost derivation

$$E \Rightarrow E + T \Rightarrow E + id_2 \Rightarrow T + id_2 \Rightarrow id_1 + id_2$$



Bottom-up parsing: relating to rightmost derivation

Rightmost derivation of $id_1 + id_2$

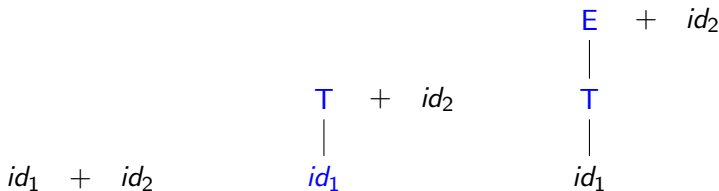
$$E \Rightarrow E + T \Rightarrow E + id_2 \Rightarrow T + id_2 \Rightarrow \underline{id_1} + id_2$$

$$\begin{array}{ccc} & T & + \quad id_2 \\ & | & \\ id_1 & + & id_2 \\ & id_1 & \end{array}$$

Bottom-up parsing: relating to rightmost derivation

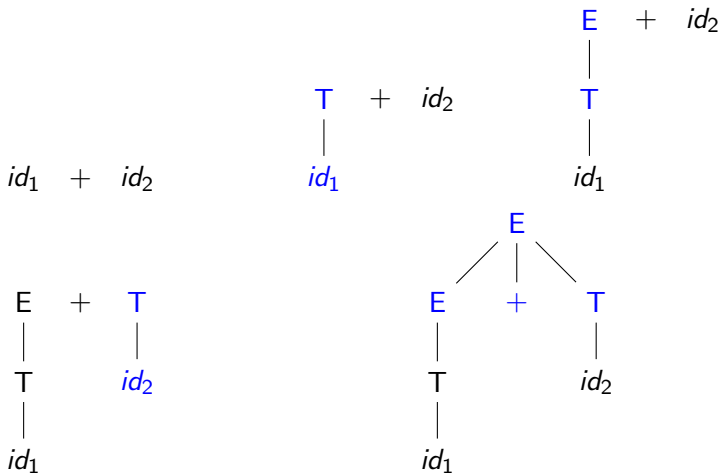
Rightmost derivation of $id_1 + id_2$

$$E \Rightarrow E + T \Rightarrow E + id_2 \Rightarrow T + id_2 \Rightarrow id_1 + id_2$$



Bottom-up parsing: relating to rightmost derivation

$$E \Rightarrow \underline{E + T} \Rightarrow E + \underline{id_2} \Rightarrow \underline{T} + id_2 \Rightarrow \underline{id_1} + id_2$$



Sentential forms in reverse order

$\underline{id_1} + id_2$ $T + id_2$ $E + id_2$ $E + T$ E

- The substring id_1 in the last sentential form $id_1 + id_2$ is replaced by T to get the previous right-sentential form $T + id_2$
 - using production $T \rightarrow id$ (body id replaced by head T)
- Reduces id to T

Sentential forms in reverse order

$$\underline{id_1} + id_2 \quad \underline{T} + id_2 \quad E + \underline{id_2} \quad \underline{E + T} \quad E$$

- the substring id_1 in the sentential form $id_1 + id_2$ is replaced by T to get the previous right-sentential form $T + id_2$
- T in $T + id_2$ is replaced by E to get $E + id_2$
- in the last step, the string $E + T$ is replaced by E

Each step replaces a substring matching the body of a production by its head.

Reduction

- **Reduction**: replaces a substring matching the body of a production by its head - the reverse of a step in a derivation.
 - Given $A \rightarrow \alpha$, substring α in a sentential form is reduced to A to get the previous sentential form.
 - The substring chosen for reduction is known as the **handle** (not the exact definition)
- The parsing process reduces the input string w to the start symbol
 - can be viewed as building a parse tree bottom-up
 - the process is known as **handle pruning**.

Exercise

Given the expression grammar

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

For each the following strings, do rightmost derivation, show the reductions and then build the parse tree bottom-up, as per the reductions.

$$id * id$$

$$(id + id) * id$$

Shift-Reduce Parsing

- A form of bottom-up parsing.
- A stack holds grammar symbols and an input buffer holds the rest of the string to be parsed.
- Parsing starts with \$ as the only symbol in stack.
- Parser **shifts** input symbols on to the stack until it gets a handle α in the stack (constituted by the topmost symbols). It then **reduces** α to the head of the appropriate production.
- The parser repeats the above step until the stack contains the start symbol and the input is empty (**accepts** the string), or until it detects an error.

Shift-Reduce Parsing

$$E \rightarrow E + T \mid T$$

$$T \rightarrow id$$

Stack	Input	Action
\$	$id_1 + id_2 \$$	shift
$\$id_1$	$+id_2 \$$	reduce by $T \rightarrow id$
$\$T$	$+id_2 \$$	reduce by $E \rightarrow T$
$\$E$	$+id_2 \$$	shift
$\$E+$	$id_2 \$$	shift
$\$E + id_2$	$\$$	reduce by $T \rightarrow id$
$\$E + T$	$\$$	reduce by $E \rightarrow E + T$
$\$E$	$\$$	accept

Four possible actions: shift/reduce/accept/error

Exercise

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

Show the configurations of a shift-reduce parser for the input $id_1 * id_2$.

Reference:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- ALSU chapter 4, section 4.5